Contents

Volume 01 — Intro, CNN, Colab labs (L00–L09) **1**
L00: Course introduction 1
L01: Introduction to Generative AI 2
L02: Activation Functions & Loss Functions in Deep Learning 3
L03: Optimizers — Part A 4
L04: Optimizers — Part B 5
L05: Convolutional Neural Network — Part A 5
L06: Convolutional Neural Network — Part B 6
L07: Introduction to Google Colab 7
L08: Practical exercise 1 — CNN on CIFAR-10 8
L09: Practical exercise 2 — Ensemble models 9

Volume 01 — Intro, CNN, Colab labs (L00–L09)

Generative AI & LLMs · NPTEL · Generative AI / LLMs **notes-3** — concept exam notes (definitions & what things do). Breadth over lab depth.

L00: Course introduction

Learning focus

- Place the 12-week course in the sequence after Foundations of Deep Learning.
- List the week-by-week theory map and the matching hands-on track.

Exam lines

- The course is theory and weekly coding, from CNN recap through RAG.
- The video order is the authority for these notes: autoencoders → VAEs → GANs → diffusion → sequence models → transformers/LLMs → RAG/ethics.
- Several topics advertised for weeks 11–12 (RAG, multimodal, ethics, hallucinations) appear in this intro; they are expanded only when a later lecture actually teaches them.

Concept tables

Weeks	Theory focus
1	Deep-learning foundations needed for generative models
2	Autoencoders and representation learning
3–4	Variational autoencoders, advanced VAEs, applications
5–6	GANs, important variants, applications
7–8	Diffusion models
9	Sequence models (RNN/LSTM) and NLP foundations
10	LLM foundations and prompt engineering

Weeks	Theory focus
11	Retrieval-augmented generation (RAG)
12	LLM capabilities, multimodal AI, ethics
Weeks	Labs announced in the intro
——:	———————————–
1–2	CNNs, transfer learning, ensemble models, then the **autoencoder** as the first generative model
3–4	Representation learning: train a **VAE** and a **conditional VAE**
5–6	Adversarial models: **DCGAN**, **cGAN**, **CycleGAN**
7–8	Diffusion: simulate **forward** noise, **reverse** denoising, and a **U-Net**
9–10	Sequence/LLM stack: **RNN**, **LSTM**, **transformer**

L01: Introduction to Generative AI

Learning focus

- Contrast discriminative models with generative models using the lecture's $P(y \mid x)$ vs $P(x)$ / $P(x, y)$ language.
- Explain why **data augmentation** and **oversampling** are not generative modeling.
- Match output type → assumed distribution → activation and loss used in code.
- Locate where a **Gaussian** shows up in VAEs, diffusion, and GANs (preview only).

Exam lines

- MLP / CNN / RNN-style predictors are discriminative: they learn $P(y \mid x)$ and **not** how x was generated.
- Generative modeling learns p_{data} so it can **sample** new realistic x (and optionally x given y).
- Augmentation and oversampling reuse existing points; they are not distribution learning.
- In code, Gaussian / Bernoulli / categorical show up as **linear+MSE**, **sigmoid+BCE**, **softmax+CCE**.
- The same Gaussian idea will reappear in VAE latents, diffusion noise, and GAN z.

Concept tables

Model	Data it fits	Typical tasks
MLP	Tabular (CSV / spreadsheet features)	Classification or regression
CNN	Images	Classification, segmentation, object detection

Model	Data it fits	Typical tasks
RNN / LSTM / GRU	Sequences / time series	"What comes next" (e.g. rainfall tomorrow from 10 days of history)
View	What is learned	When you use it
——	———————–	———————–
Unsupervised generation	$P(x)$ or $p_\theta(x) \approx p_{\text{data}}(x)$	Features only (no labels)
Joint modeling	$P(x, y)$	Features **and** labels
Conditional generation	$P(x \mid y)$ (lecture language: generate x given a chosen class y)	"Only digit 7", "only smiling faces", extra **malignant** medical images

---

L02: Activation Functions & Loss Functions in Deep Learning

Learning focus

- State why a neuron needs a nonlinear activation, and where activations sit (hidden vs output).
- Compute ReLU / Leaky ReLU / ELU on a weighted sum, and pick sigmoid, tanh, or softmax at the output.
- Map this course's models (CNN, autoencoder, VAE, GAN, diffusion, LLM) to typical hidden and output activations.
- Distinguish reconstruction / classification losses from generative-model losses (KL, GAN, noise-prediction).

Exam lines

- No activation ⇒ linear neuron; ReLU-family activations also **gate** negative features.
- Hidden menu: ReLU (dying neurons) → Leaky ReLU (0.01) / PReLU (learned slope) → ELU (smooth exponential) → GELU (transformers) / SiLU (diffusion).
- Output menu: sigmoid $\{0, 1\}$, tanh $\{-1, +1\}$, softmax (multiclass / next token).
- Generative reconstruction uses MSE or BCE on x **vs** $\hat{x}$, not on class labels; LLMs use (sparse) categorical CE; VAEs add KL; GANs and diffusion have their own losses.

Concept tables

If there is no activation	What goes wrong
The weighted sum is emitted unchanged	The neuron is a **linear** model
Stacking linear layers stays linear	The net cannot learn **complex patterns**
No ReLU-family gate	Negative / unused features are not suppressed
Weighted sum x	$\max(0, x)$

If there is no activation	What goes wrong
———————	————
negative (e.g. -3)	0
0	0
positive (e.g. 4)	4

L03: Optimizers — Part A

Learning focus

- Place the optimizer in the train loop: forward pass → loss → gradients → weight update.
- Apply $W_{\text{new}} = W_{\text{old}} - \eta\, \partial L/\partial W$ and explain the minus sign.
- Contrast **SGD**, **mini-batch SGD**, and **batch gradient descent** by when weights are updated.
- Write SGD **with momentum** as an exponentially weighted moving average of gradients.

Exam lines

- Optimizer = use $\partial L/\partial W$ to raise or lower W so loss falls; the minus sign is "move downhill."
- SGD / mini-batch / batch GD differ only in **how many samples** feed one update.
- SGD zigzags because each update sees only the current g_t.
- Momentum replaces g_t by an EWMA v_t ($\beta \approx 0.9$): same-direction gradients **speed up**; a sudden sign flip **damps** rather than snaps.

Concept tables

Gradient $\partial L/\partial W$	Arithmetic	W_{new}	Effect
-2 (negative)	$5 - 0.1(-2) = 5.2$	5.2	**weight increases**
positive	5 minus a positive quantity	< 5	**weight decreases**
Optimizer	Samples per update	Typical path to the minimum	
———–	———————–	———————————–	
SGD	1	noisy, much zigzag	
Mini-batch SGD	k (hyperparameter)	smoother than SGD	
Batch GD	all n	one step per full pass	

L04: Optimizers — Part B

Learning focus

- Separate **gradient sign** (direction) from **gradient magnitude** and **learning rate** (step size).
- Explain why a **single** η for millions of weights is a bad idea.
- Write **AdaGrad**'s per-parameter rate from the sum of **squared** past gradients, and state why the rate can die.
- Show how **RMSProp** replaces that sum by an EWMA of g_t^2, and how **Adam** = momentum + RMSProp.

Exam lines

- Sign of g = direction; $|g|$ and η = step size. A huge η with a huge g can slam W (e.g. $5 \to 0$).
- Weights see different $|g|$; one global η is the SGD assumption we drop.
- AdaGrad: $\eta_t = \eta/\sqrt{\sum g^2 + \varepsilon}$ — good adaptivity, **vanishing rate** as the sum grows.
- RMSProp: same denominator idea with **EWMA of** g^2 so old squares fade.
- Adam: **momentum (**β_1**) + RMSProp (**β_2**)**.

Concept tables

Situation	What happens to W
Large $\|g\|$	**Large** change (lecture: $5 \to 4$)
Small $\|g\|$	**Tiny** change (lecture: $5 \to 4.95$)
η	Qualitative result
——–	——————–
small	small change in W
moderate	e.g. $5 \to 4$
high	**abrupt** jump — lecture's scare case: $5 \to 0$

L05: Convolutional Neural Network — Part A

Learning focus

- Describe how a machine stores an image (pixels, height × width, channels).
- List why a fully connected MLP is the wrong first model for images.
- Name CNN applications (classification, detection, segmentation variants).
- Run one **convolution**: kernel, element-wise multiply-and-sum, stride, padding, output size, Keras `Conv2D` arguments.

Exam lines

- Images are $H \times W \times C$ pixel grids; flattening them for an MLP drops neighbourhoods and explodes parameters.
- CNN applications: classify, detect, semantically / instance-segment (sheep, CT tumour, ...).
- A kernel slides, multiply-and-sums, writes a feature map; **many kernels ⇒ many maps**; kernel weights are learned.
- Size: $\lfloor (n + 2p - f)/s \rfloor + 1$, floor if needed; **padding** protects borders; **stride** is how far the kernel jumps.
- Keras: `Conv2D`, `padding="same"|"valid"`, default stride 1.

Concept tables

Image	Channels	Flattened length
28×28 colour	3 (RGB)	$28 \times 28 \times 3 = 2352$
200×200 colour	3	120,000

Task	Lecture example
——	——————–
Image classification	sheep or not; **benign vs malignant** / cancerous vs not
Object detection	several objects; **where** each one is
Image segmentation	keep only the **region of interest**
Classification + **localization**	same class (sheep) **and** a box
Semantic segmentation	road / grass / sheep — one colour per **class**
Instance segmentation	sheep 1, sheep 2, sheep 3 — same class, **different instances**

L06: Convolutional Neural Network — Part B

Learning focus

- Apply **ReLU** to a feature map and explain why it does **not** downsample.
- Count **learnable conv parameters** and the **output tensor size** after conv.
- Run **max / average / min / max-average-min** pooling, including Keras `MaxPooling2D`.
- Stack conv–ReLU–pool, **flatten**, and a **fully connected** classifier (sigmoid vs softmax).

Exam lines

- ReLU after conv zeros negatives and **keeps spatial size**; pooling is the downsampler.
- Conv parameters: $(f_H f_W C_{\text{in}} + 1) \times n_{\text{filters}}$; output spatial size $\lfloor (I - F + 2P)/S \rfloor + 1$ with depth = number of filters.
- Pooling: 2×2 window, default stride 2; max (salient), average, min, or max-average-min; Keras `MaxPooling2D`.
- Stack blocks until the map is small enough; flatten; dense head with C outputs (sigmoid vs softmax).

Concept tables

$f_H \times f_W$	C_{in}	Filters	Arithmetic	Parameters
5×5	3	6	$(25 \times 3 + 1) \times 6$	456 (stated)
3×3	3	64	$(9 \times 3 + 1) \times 64$	1792
5×5	64	128	$(25 \times 64 + 1) \times 128$	204,928

Task	Output neurons	Activation
——	—————	————
Binary	1	**sigmoid**
3-class	3	**softmax**

L07: Introduction to Google Colab

Learning focus

- Open Colab, create a notebook, connect a runtime, and pick CPU / GPU / TPU.
- Run **code** vs **text** cells (`Shift+Enter`).
- Pull a dataset from **Kaggle**, **GitHub**, or **UCI**, land it in **Google Drive**, and paste the path into pandas / folder variables.

Exam lines

- Colab = free cloud Jupyter; connect, then set **Python 3** and **CPU/GPU/TPU** (GPU for long jobs, TPU for huge / NLP).
- Data path: Kaggle / GitHub / UCI → disk → **Google Drive** → mount → **Copy path** into `read_csv` or train/test variables.
- Kaggle also runs notebooks; GitHub also hosts full CNN projects (confusion matrices included).

Concept tables

Field	Options the lecture shows	What to pick
Runtime type	**Python 3**, **R**, **Julia**	Python 3 for this course; R if you are writing R
Hardware accelerator	**CPU**, **GPU**, **TPU**	see below
Runtime version	several	**latest**
Accelerator ——————-	When ——	
CPU (default)	fine for small demos; lecture selects CPU "for the time being"	
GPU (graphics processing unit)	same code that might take **10–12 hours** on CPU can take **1–2 hours** on GPU	
TPU (tensor processing unit)	**very large** datasets or **NLP**	

L08: Practical exercise 1 — CNN on CIFAR-10

Learning focus

- Load **CIFAR-10**, normalize pixels to $[0, 1]$, and plot a 2×5 sample grid.
- Build a **three-block** Keras CNN (32 / 64 / 128 filters), flatten, dense, **dropout** 0.5, softmax-10.
- Compile with **Adam** + **sparse categorical cross-entropy** + **accuracy**; train **10 epochs**, batch **64**, `validation_split=0.2`.
- Read accuracy/loss curves, test metrics, per-image predictions, and the 10×10 **confusion matrix**.

Exam lines

- CIFAR-10: $50\text{k} + 10\text{k}$ RGB 32×32 images, 10 named classes; pixels $/255$.
- CNN: Conv32–pool, Conv64–pool, Conv128, flatten, Dense128 ReLU, **Dropout** 0.5, Dense10 softmax.
- Adam + **sparse** CCE (integer labels) + accuracy; 10 epochs, batch 64, 20% val.
- This run: train $\sim 75\%$, val $\sim 72\%$, test $\sim 71\%$ / loss 0.82; CM diagonal (e.g. 815, 839) vs off-diagonal errors (35, 16, ...).

Concept tables

Import	Why
`tensorflow as tf`	DL framework
`tensorflow.keras`: **`datasets`**, **`layers`**, **`models`**	CIFAR loader, layer types, Sequential
`matplotlib.pyplot as plt`	plots
`numpy as np`	arrays
`sklearn.metrics`: **`confusion_matrix`**, **`confusion_matrix_display`**	true vs predicted counts
Split	Contents
——-	——-
`x_train`	training **images**
`y_train`	training **labels**
`x_test`	test images
`y_test`	test labels (ground truth to compare with $\hat{y}$)

L09: Practical exercise 2 — Ensemble models

Learning focus

- Contrast **serial** vs **parallel** ensembling; implement **three CNNs in parallel** and **average** their probability vectors.
- Differentiate the three members: plain CNN, CNN + **batch normalization**, CNN + **dropout** 0.5.
- Train each **5 epochs** (batch 64, `validation_split=0.2`, Adam, sparse CCE) and compare **individual val acc** to **ensemble test acc** $\approx 71\%$.

Exam lines

- Parallel ensemble: three CIFAR CNNs, **average softmax vectors**, then argmax.
- Members: (1) two-block CNN, (2) same idea + **batch norm** ($\frac{x-\mu}{\sigma}$), (3) CNN + 50% **dropout**.
- Train **5** epochs, batch **64**, val 20%, Adam, **sparse** CCE, accuracy.
- This run: val $67\%/68\%/61\%$; ensemble test 71.14% — fewer mistakes than any one model.

Concept tables

Import	Role
`tensorflow as tf`	build nets
Keras `datasets`, `layers`, `models`	CIFAR-10, layers, Sequential
`matplotlib.pyplot as plt`	plots
`numpy as np`	arrays
sklearn `confusion_matrix`, `confusion_matrix_display`	true vs predicted

Import	Role
later: `accuracy_score` from sklearn	ensemble accuracy

Split	Samples
Train	50,000
Test	10,000
